HTR-ground

Web-based correction tool for HTR (Handwritten Text Recognition) output — for producing *ground truth* transcriptions of historical manuscripts.

The user sees the page image on the left, the recognised text lines on the right, and edits the text in visual context. Save exports ALTO XML, PAGE XML, or the internal JSON.

The name comes from *ground truth* — the verified reference data used in machine learning, which is precisely what this tool helps produce.

Two usage modes

- **Demo / playground** (`/demo` , public) — upload your own image + annotation pair from disk, correct, download the result in JSON / ALTO XML / PAGE XML / searchable PDF. Works even from `file://` (client-side JSON only).
- **Projects** (`/projects` , login required) — browse a server-side `projects/` folder tree, open image+transcript pairs directly, save-in-place, set per-file statuses, see who else is currently editing what. Per-user login with per-project visibility rules. Ideal when many people work on a shared corpus.

Project layout

```
HTR-ground/
├── frontend/
│   ├── landing.html            # home: two cards (Demo / Projects), Bootstrap
│   ├── login.html              # shared-password login page, Bootstrap
│   ├── editor.html             # DOM skeleton (toolbar + two panels)
│   ├── editor.css              # all editor styles (dark theme)
│   ├── editor.js               # editor logic: load, zoom, filters, save
│   ├── projects.html           # Projects file browser skeleton, Bootstrap
│   ├── projects.css            # Projects dark-theme styles on top of Bootstrap
│   └── projects.js             # folder navigation, breadcrumb, listing
│
├── backend/
│   ├── conf/
│   │   └── auth_default.json   # empty template (users/projects/session
│   │                           #   config). conf/auth.json is gitignored
│   │                           #   and generated by `python -m app.users bootstrap`.
│   └── app/
│       ├── main.py             # FastAPI app: routes, session middleware
│       ├── auth.py             # bcrypt auth, session guards, per-user login
│       ├── users.py            # CLI: bootstrap/add/remove/list/set-password/promote/demote
│       ├── acl.py              # per-project visibility (longest-prefix match)
│       ├── projects.py         # safe path resolve, pair detection, load/save
│       ├── meta.py             # per-file `.htrground-meta.json` sidecar (status + audit)
│       ├── presence.py         # in-memory presence tracker (who is editing what)
│       ├── schema.py           # Page / Region / Line Pydantic models
│       └── converters/
│           ├── __init__.py     # detect_format + convert + export dispatch
│           ├── alto.py, page.py, htr_json.py       # import
│           ├── to_alto.py, to_page.py              # export
│           └── to_pdf.py                            # searchable two-layer PDF
│
├── projects/                   # (user content) shared corpus tree
├── fonts/EBGaramond-Regular.ttf # required for PDF export
└── README.md
```

Setup

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# First-run: create conf/auth.json with an admin user (password printed once)
python3 -m app.users bootstrap --generate

uvicorn app.main:app --reload --port 8000
```

Open http://localhost:8000. The landing page has two cards:

- **Demo** — no login. Upload an image + annotation, edit, download.
- **Projects** — redirects to login on first visit. Then browse `projects/`.

Tests: `pytest -q` from the `backend/` folder.

Docker deployment

For deploying on an older server (or anywhere Docker runs), the repo ships a `Dockerfile` and `docker-compose.yml`. Config and data are bind-mounted from `./docker-data/` so backup is one `tar` command away.

By default the mounts point at `./docker-data/` next to the repo:

```
docker-data/
├── conf/                    # auth.json (created by bootstrap)
└── projects/                # your corpus (images + annotations + sidecars)
```

If you want them elsewhere (e.g. `/srv/htr-ground/`), copy `.env.example` to `.env` next to the compose file and override:

```
cp .env.example .env
# then edit:
#   HTR_CONF_DIR=/srv/htr-ground/conf
#   HTR_PROJECTS_DIR=/srv/htr-ground/projects
```

The PDF export font is baked into the image; only override it with a mount if you want a different typeface.

First run

```
# 1. Create the mount directories (adjust paths if you set them in .env)
mkdir -p docker-data/{conf,projects}

# 2. Build the image
docker compose build

# 3. Bootstrap the admin user — the password is printed once
docker compose run --rm app python -m app.users bootstrap --generate

# 4. Start the server
docker compose up -d
```

Open http://localhost:8000 — or your server's URL if remote.

User management inside the container

```
docker compose exec app python -m app.users list
docker compose exec app python -m app.users add anna "Kovács Anna" --generate
docker compose exec app python -m app.users set-password anna --generate
```

Backup

Everything persistent lives under `docker-data/`. To back up:

```
tar czf htr-ground-backup-$(date +%Y%m%d).tgz docker-data/
```

Restore = untar to the same layout, then `docker compose up -d`.

HTTPS / reverse proxy

The compose file ships with the container bound to `127.0.0.1:8000` — so nothing is exposed to the outside directly. Put nginx / Apache / Caddy / Traefik in front for TLS termination.

The compose file also sets `HTR_GROUND_HTTPS=1`, which enables the `Secure` flag on the session cookie so it never leaks over plain HTTP. If you're testing locally without HTTPS, remove this env var (otherwise the browser will refuse to send the cookie back).

nginx — minimal snippet, drop it in a `server` block:

```
server {
    listen 443 ssl http2;
    server_name htr.example.com;

    ssl_certificate     /etc/letsencrypt/live/htr.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/htr.example.com/privkey.pem;

    # File uploads / PDF exports can be large
    client_max_body_size 100M;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# Redirect HTTP → HTTPS
server {
    listen 80;
    server_name htr.example.com;
    return 301 https://$host$request_uri;
}
```

Apache — needs `mod_proxy`, `mod_proxy_http` and `mod_ssl` enabled:

```
<VirtualHost *:443>
    ServerName htr.example.com

    SSLEngine on
    SSLCertificateFile    /etc/letsencrypt/live/htr.example.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/htr.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass        /  http://127.0.0.1:8000/
    ProxyPassReverse /  http://127.0.0.1:8000/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

The Docker container is configured with `uvicorn --forwarded-allow-ips=*`, so it trusts the `X-Forwarded-Proto` header from any source. This is safe here because the container is only reachable from localhost (via the port binding above), never from the public network directly.

Presence and workers

Presence is in-memory in the FastAPI process. The compose config uses a single worker (`--workers 1`) — do not increase this without moving the tracker to Redis or similar shared store. For a small team on one server, a single worker is plenty.

Managing users

Everything happens via `python -m app.users`. There is no admin UI — the password store is a flat JSON that only the CLI writes.

```
python -m app.users bootstrap [--username admin] [--generate]  # first run
python -m app.users add <username> [display] [--admin] [--generate]
python -m app.users set-password <username> [--generate]
python -m app.users list
python -m app.users remove <username>
python -m app.users promote <username>    # grant admin
python -m app.users demote <username>     # revoke admin
```

Passwords are never stored in plaintext — the CLI writes bcrypt hashes. When `--generate` is used, a strong 4×4 alphanumeric password (e.g. `k7Qm-vXn9-pR2t-Lb8H`) is printed **once** and never shown again. If you forget it, run `set-password` to reset.

Data flow

Internal `Page` model

Both frontend and backend speak the same JSON structure (Pydantic models in `backend/app/schema.py`):

```
{
  "regions": [
    {
      "coords": [[x, y], ...],
      "rect":   [x, y, w, h],
      "lines": [
        {
          "coords":   [[x, y], ...],
          "rect":     [x, y, w, h],
          "baseline": [[x1, y1], [x2, y2]],
          "text":     "..."
        }
      ]
    }
  ],
  "image_width": 5496,
  "image_height": 3670,
  "source_format": "alto-xml"
}
```

Coordinates are always in the source image's native pixel space.

Import (`converters/`)

`POST /api/convert` autodetects the input format (`.json`, ALTO XML, PAGE XML) and returns a `Page`. Format detection is done from the first few KB of the byte stream, not the filename.

- ALTO XML — if `<Shape><Polygon POINTS="…"/></Shape>` is present it is used; otherwise the polygon is synthesised from `HPOS/VPOS/WIDTH/HEIGHT`.
- PAGE XML — polygons come from `<Coords points="…"/>` directly.
- Native JSON — passthrough, validated against the `Page` schema.

Export (`converters/to_alto.py`, `to_page.py`)

Reverse direction, called by `POST /api/export` (demo mode) and `PUT /api/project-file` (project mode). Both formats emit real polygons, not just bounding boxes.

Round-trip guarantee

`convert(export(page, format), format=…)` returns the same regions/lines/ polygons/text. Verified for `alto-xml` and `page-xml` against the Bakonykuti sample (68 lines, exact polygon match).

Projects

The **Projects** mode lets multiple users work against a shared corpus on the server without uploading files through the browser. The tree can be arbitrarily deep — folders that contain image + annotation pairs are the leaves.

Adding a project

Drop files into `projects/<project>/<subproject>/…`:

```
projects/
├── Minta/                      # committed as a demo (see .gitignore)
│   ├── 1949/
│   │   ├── Minta_V1_049.jpg
│   │   └── Minta_V1_049.json
│   └── 1950/
│       ├── Minta_V1_050.jpg
│       └── Minta_V1_050.alto.xml
└── …                           # your own projects, gitignored by default
```

By default `.gitignore` excludes everything under `projects/` except for the `Minta/` sample folder, which is committed as a working demo. Your own projects stay local unless you explicitly add them.

Pair detection groups files by basename, respecting compound extensions (`.alto.xml` and `.page.xml`). Image extensions: `.jpg`, `.jpeg`, `.png`, `.tif`, `.tiff`, `.gif`. Annotations: `.json`, `.xml`, `.alto.xml`, `.page.xml`.

If both an image and an annotation with the same basename exist, they show up as a single pair in the browser. Missing halves show up with a `nincs kép` ("no image") or `nincs átirat` ("no transcript") tag.

`.htrground.json` — save format per folder

Optional. Only looked at in leaf folders. Controls what format the **Save** button writes when a file is edited in that folder:

```
{
  "save_format": "alto-xml"
}
```

Valid values: `json`, `alto-xml`, `page-xml`. If the file is missing or invalid, the original loaded format is used. The **Export** dropdown still lets you download in any of the three formats regardless of this setting.

Path safety

All path handling goes through `projects.resolve_safe` which resolves the user-provided path relative to `projects/` and refuses any escape (`../`, absolute paths, symlink chains that leave the tree). Attempted escapes raise `PathEscapeError` → HTTP 400.

Authentication

Per-user model. Each user has a `username`, a bcrypt `password_hash`, and an optional `is_admin` flag. Fits a small trusted team without needing an admin UI: the CLI manages the flat JSON file.

- Users, ACL and session config live in `backend/conf/auth.json` (gitignored). Template is `backend/conf/auth_default.json` in the repo (empty).
- Session is a signed cookie via `starlette.middleware.sessions.SessionMiddleware`, keyed with the `session_secret` from `auth.json`.
- Public routes: `/`, `/demo`, `/login`, `/editor.*`, `/api/convert`, `/api/export`, `/api/export-pdf`, `/api/health`, `/api/session`, `/api/status-values`.
- Auth-gated: `/projects*`, `/api/projects*`, `/api/project-*`, `/api/presence*`.

Logout: `POST /logout` clears the session.

Per-project visibility (ACL)

The `projects` map in `auth.json` restricts access. If a path is not listed, **everyone** sees it. If it is listed, only the users in `visible_to` do. The most-specific (longest-prefix) entry wins. Admin users bypass ACL.

```
"projects": {
  "Titkos":           { "visible_to": ["anna", "lektor"] },
  "Bakonykuti":       { "visible_to": ["anna", "bela"] },
  "Bakonykuti/part2": { "visible_to": ["anna"] }
}
```

- `Bakonykuti/part1/1949` → inherits `Bakonykuti` → anna & bela
- `Bakonykuti/part2/inner` → matches `Bakonykuti/part2` → anna only
- `Nyilvanos` (not listed) → everyone
- `"visible_to": ["*"]` means every authenticated user

API reference

Base URL: `http://<host>:<port>` (default 8000). All auth-gated endpoints return HTTP 401 as JSON if the session cookie is missing.

Public

Method	Path	Purpose
GET	`/`	Landing page
GET	`/demo`	Demo editor (upload-based)
GET	`/login`, POST `/login`	Login form + username/password submit
POST	`/logout`	Clears session
GET	`/api/session`	`{authenticated, username?, display_name?, is_admin?}`
GET	`/api/health`	`{status: "ok"}`
POST	`/api/convert`	HTR file → internal `Page` JSON
POST	`/api/export`	`Page` JSON → JSON / ALTO / PAGE download
POST	`/api/export-pdf`	`Page` + image → searchable two-layer PDF
GET	`/api/status-values`	UI dropdown source: `{values, default}`

Auth-gated

Method	Path	Purpose
GET	`/projects`, `/projects/{path}`	Projects browser HTML (deep-link OK)
GET	`/projects/edit?path=…&basename=…`	Editor in project mode
GET	`/api/projects`, `/api/projects/{path}`	Folder listing (JSON) — includes `meta` + `presence`
GET	`/api/project-file?path=…&basename=…`	Load one pair → `{page, image_url, save_format, meta, …}`
PUT	`/api/project-file?path=…&basename=…`	Save one pair in-place; auto-records `edited_by/at`
GET	`/api/project-image?path=…&basename=…`	Serve the pair's image file
PUT	`/api/project-status?path=…&basename=…`	Set status; body: `{status, notes?}`
POST	`/api/presence/heartbeat`	Client tells the server "I'm still here"
POST	`/api/presence/leave`	Explicit leave (browser `beforeunload`)
GET	`/api/presence?path=…&basename=…`	Who else is currently on this file

Example: convert + export

```
# Upload an ALTO XML, get internal JSON back
curl -F "file=@examples/Bakonykuti_V1_049.alto.xml" \
     http://localhost:8000/api/convert

# Export a Page as PAGE XML
curl -X POST http://localhost:8000/api/export \
     -H "Content-Type: application/json" \
     -d '{"page": {...}, "format": "page-xml", "basename": "Bakonykuti_V1_049"}' \
     -o out.page.xml
```

Frontend architecture

Three-file split, no build step:

- **`editor.html`** — DOM skeleton with a `<base href="/">` injected via inline script when served over HTTP. This makes relative `editor.css` / `editor.js` resolve to the server root regardless of the current URL (needed because `/projects/edit?…` would otherwise resolve them to `/projects/editor.css`, which falls into the folder-browser catch-all and returns HTML). In `file://` mode the base tag is skipped and relative paths resolve normally.
- **`editor.css`** — dark theme, all layout and component styles.
- **`editor.js`** — one top-level module: state, DOM refs, image filters, file loading, zoom logic, text-panel rendering, SVG overlay, detail view (canvas crop with contrast-aware highlighting), save + export.

`editor.js` detects the mode at startup by looking at `window.location.pathname` — anything under `/projects/edit` puts it in project mode. Project mode:

- Hides upload buttons, shows a project-info label (`path / basename / fmt`).
- Auto-loads the pair via `GET /api/project-file`.
- Save button = in-place `PUT /api/project-file`.
- Separate **Export ▾** button opens the format dropdown for downloads.
- Back button in the toolbar returns to the parent folder in the browser.

The Bootstrap-based pages (landing, login, projects) share a minimal dark theme layered on Bootstrap 5.3 from CDN.

PDF export (searchable / two-layer)

`POST /api/export-pdf` builds a searchable PDF by combining the image with an invisible text layer positioned at each line's baseline. The PDF text is extractable with `pdftotext` and is selectable in any viewer, but stays visually hidden behind the image.

Requires a Unicode TrueType font. The default location is `fonts/EBGaramond-Regular.ttf`; override via the `HTR_GROUND_FONT` env var. Download the default:

```
mkdir -p fonts && curl -L -o fonts/EBGaramond-Regular.ttf \
    https://github.com/octaviopardo/EBGaramond12/raw/master/fonts/ttf/EBGaramond-Regular.ttf
```

If the font is missing, the endpoint returns HTTP 500 with a clear message — the other export formats (JSON / ALTO / PAGE) work regardless.

The image is downscaled to 1500 px width and re-encoded as JPEG (quality 78) before embedding, so page PDFs typically fall in the 100–500 kB range even for high-resolution scans.

Statuses (per-file)

Each pair has an implicit status. The list is hardcoded to keep the UI predictable: `"új"`, `"folyamatban"`, `"ellenőrzésre vár"`, `"kész"`. The default when no sidecar exists is `"új"`, so **you don't have to register anything when adding a new folder** — files just appear with the default status.

Statuses live in a sidecar next to the pair:

```
projects/<...>/Bakonykuti_V1_049.jpg
projects/<...>/Bakonykuti_V1_049.json
projects/<...>/Bakonykuti_V1_049.htrground-meta.json    ← sidecar
```

Sidecar shape:

```
{
  "status":            "folyamatban",
  "status_changed_by": "anna",
  "status_changed_at": "2026-07-09T14:23:00Z",
  "edited_by":         "anna",
  "edited_at":         "2026-07-09T14:30:00Z",
  "notes":             ""
}
```

- `status_*` is updated only when someone explicitly changes the status (via the badge dropdown in the file list).
- `edited_*` is updated automatically on save (`PUT /api/project-file`).
- The two are independent so a reviewer can change status without touching the content, and an editor can save without changing status.

Sidecars are written atomically (`.tmp` + `replace()`). They are not listed as annotations — the pair detector explicitly skips them.

Presence

The projects list and the editor show who else is currently working on a file. Presence is **not persisted** — the tracker is in-memory in the FastAPI process. Process restart wipes it, which is fine.

- Client (editor in project mode) sends `POST /api/presence/heartbeat` every 25 seconds while the tab is open.
- Server considers a user "gone" after 60 seconds with no heartbeat.
- On editor load, `GET /api/presence` — if someone else is active, the UI confirms "Anna is editing this — sure you want to continue?".
- No hard lock — the last save wins. The confirmation exists to prevent accidental clobbers, not enforce serialization.
- Projects list auto-refreshes every 20 seconds (only when the tab is visible) so the presence badges stay live.

If you ever run multiple workers (e.g. `gunicorn --workers 4`), the in-memory tracker won't share state across processes. Switch to Redis or similar at that point.

Development notes

- **Config**: `backend/conf/auth.json` holds users + per-project ACL + session config. `auth_default.json` is a committed empty template.
- **Adding a new format** (e.g. hOCR): add a `parse()` in `converters/x.py`, extend `detect_format()` and the dispatch table in `converters/__init__.py`. For export, add `to_x.py` and register it in `EXPORT_MIME` / `EXPORT_EXT`.
- **Route ordering**: static asset routes (`/editor.css`, `/projects.css`, etc.) are declared *before* the `/projects/{deep_path:path}` catch-all so they win the match.
- **ALTO polygons**: the exporter always writes `<Shape><Polygon POINTS>` — no bare bounding-box output. This means round-tripping through HTR-ground never loses polygon information, even if the input source (e.g. some legacy ALTO producers) only had rectangles.
- **Atomic writes**: `save_pair()`, `meta._write_atomic()` and `auth.save_auth_config()` all write to `<name>.tmp` first, then `Path.replace()` renames — safe against partial writes.
- **Test isolation**: the pytest conftest builds an isolated `auth.json` in a tmp folder *at module import time* (not in a fixture) because `SessionMiddleware` reads `session_secret` at import.

Author

István Szekrényes

License

HTR-ground is free software: you can redistribute it and/or modify it under the terms of the **GNU General Public License version 3** (or, at your option, any later version) as published by the Free Software Foundation.

This program is distributed in the hope that it will be useful, but **without any warranty**; without even the implied warranty of merchantability or fitness for a particular purpose. See the LICENSE file for the full text of the GNU General Public License, or visit https://www.gnu.org/licenses/gpl-3.0.html.